



INDIVIDUAL ASSIGNMENT

CT046-3-M

APPLIED MACHINE LEARNING

CSSE___CT046-3-M-AML-L-1___2026-01-26

HAND OUT DATE: 05 FEBRUARY 2026

HAND IN DATE: 01 MAY 2026

WEIGHTAGE: 100%

INSTRUCTIONS TO CANDIDATES:

- 1 Submit your assignment via Moodle.**
- 2 Students are advised to underpin their answers with the use of references (cited using the 7th Edition of APA Referencing Style).**
- 3 Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**
- 4 Cases of plagiarism will be penalized.**
- 5 You must obtain 50% overall to pass this module.**

Performance Benchmarking of Ensemble Learning Techniques for Network Intrusion Detection

Prepared by: Masrie bin Bukhori

TP Number: TP143355

Table of Contents

1. Introduction	6
2. Related Works.....	8
2.1 Introduction to NSL-KDD and IDS Evolution.....	8
2.2 Thematic Review: Preprocessing and Data Imbalance	9
2.3 Thematic Review: Ensemble Learning Benchmarks.....	10
2.4 Summary Table and Gap Analysis	12
PART B.....	14
3. Methods	14
4. Dataset Preparation	16
4.1 Dataset Overview	16
4.2 Exploratory Data Analysis (EDA)	17
Dataset Preprocessing and Cleaning.....	20
5. Model implementation	23
6. Model Validation	25
7. Analysis & Recommendations.....	28
Conclusion	30
Reference.....	31
Acknowledgement.....	32

Abstract

Network intrusion detection is one of the important aspects of network security, and the primary goal of intrusion detection is to identify malicious activities in the network traffic and prevent them before the occurrence of any major damage. Traditional intrusion detection techniques have limitations in dealing with high-dimensional and complex data sets. In this study, the performance of ensemble learning techniques in intrusion detection using the NSL-KDD data set is presented. The NSL-KDD data set is considered one of the best data sets for intrusion detection, and it provides realistic data about the network traffic. The data set also avoids the problem of over-cleaned data sets. In this study, the performance of the ensemble learning techniques Random Forest, AdaBoost, and Gradient Boosting in intrusion detection using the NSL-KDD data set is presented. The performance of the models is also compared using different performance metrics. The performance of the models is also presented in the form of accuracy, precision, recall, and F1-score. The study demonstrates the performance of the ensemble learning techniques in intrusion detection using the NSL-KDD data set.

1. Introduction

Network intrusion detection is one of the most important aspects of computer security that needs to be addressed. In today's networks, intrusion detection systems must deal with huge amounts of data traffic, which makes volume and velocity characteristics of Big Data very evident in the sense that large streams of data keep on coming and need to be analysed fast enough. As the complexity of computer networks is on the rise, traditional methods of intrusion detection, such as signature-based detection systems, are no longer effective in detecting intrusions because of their inability to deal with high dimensionality and complexity of data, which is indicative of the diversity of Big Data, where traffic within the network includes many features like protocols, connections, and behaviour.

Machine learning has been found to be one of the most viable approaches to intrusion detection, especially when it comes to big data, because it can learn from big network data. Among all the different types of machine learning algorithms that can be used for intrusion detection, ensemble learning has shown promising results for detecting intrusions in computer networks compared to other types of learning algorithms. Ensemble learning algorithms such as Random Forest, AdaBoost, and Gradient Boosting can be used for intrusion detection on the NSL KDD dataset.

The NSL-KDD dataset is considered one of the major benchmarks in the field of intrusion detection. The dataset contains realistic data and attempts to resolve the issues of the older dataset, in which there were redundant data entries and the data had become overly clean.

The objective of this study is to evaluate the performance of different ensemble learning techniques for intrusion detection in the context of the NSL-KDD dataset.

The specific objectives of the study are:

- Preprocess the NSL-KDD dataset and make it ready for the models.
- Implement the ensemble learning models on the NSL-KDD dataset.
- Use different performance metrics to evaluate the performance of the models.
- Compare the performance of the models and determine the best ensemble learning technique by looking at the metrics.

Despite the fact that some limitations were established in the literature, the NSL-KDD dataset has been chosen for the experiment based on the suitability for the experimental evaluation of machine learning algorithms in the context of intrusion detection. While the dataset might not fully reflect the latest network traffic characteristics, it has been recognized as a structured benchmark for the evaluation and comparison of different machine learning algorithms. Besides, the dataset has a mix of categorical and numerical features, a reasonable number of attributes, and challenges such as class imbalance and redundant features. These factors have been found to be appropriate for the context of the assignment, where the application of different machine learning models, preprocessing strategies, and critical evaluation has been emphasized.

2.Related Works

2.1 Introduction to NSL-KDD and IDS Evolution

The evolution of intrusion detection system (IDS) datasets has been a continuous process to overcome the limitations and deficiencies in the initial benchmark datasets. For instance, the KDD Cup 1999 dataset was the first benchmark dataset for the evaluation of intrusion detection system models. However, the KDD Cup 1999 dataset was later identified to have a few deficiencies. Tavallaee et al. (2009) presented a comprehensive study on the KDD Cup 1999 dataset and identified a few limitations. For instance, the dataset was found to have a high percentage of redundant instances. Specifically, the study found that 78 percent of the training instances and 75 percent of the testing instances were redundant.

In order to overcome these challenges, a new dataset named NSL-KDD was developed as a refined version of KDD'99. This dataset eliminates duplicates and offers a balanced representation of samples. Labhalla and Battou (2026) emphasize that NSL-KDD is still used as a prominent dataset for research purposes owing to its organized structure and potential for facilitating fair comparison between various models. However, there are still challenges in detecting highly imbalanced User-to-Root (U2R) and Remote-to-Local (R2L) attacks in the NSL-KDD dataset.

Recent studies have emphasized that although NSL-KDD solves the problem of redundancy, it does not entirely represent the complexity of network traffic. Sambasivam (2025) explains that NSL-KDD does not include contemporary attack patterns like zero-day attacks and encrypted attacks, which are commonly used in the present-day environment of cyber security. Sideek et al. (2026) explain that contemporary datasets like NF-UQ-NIDS have included different datasets in the form of NetFlow data, which is helpful in representing the complexity of network environments, especially in the context of Internet of Things (IoT).

Recent studies have criticized NSL-KDD on the basis of the fact that NSL-KDD uses simple categories of attacks, which are outdated. Adversarial robustness analysis (2025) explains that NSL-KDD does not represent contemporary attacks, which have been generated based on contemporary databases of vulnerabilities. This is because contemporary datasets like UNSW-NB15 and CIC-IDS2017 have included different

types of traffic patterns, which are helpful in representing contemporary intrusion detection systems.

Despite these limitations, NSL-KDD continues to be widely used as a benchmark dataset due to its controlled structure and suitability for comparative analysis. Its balanced composition allows researchers to evaluate algorithmic performance under standardized conditions. As a result, NSL-KDD remains relevant for academic benchmarking, while more recent datasets are increasingly preferred for evaluating practical, real-world intrusion detection systems.

2.2 Thematic Review: Preprocessing and Data Imbalance

One of the major challenges in intrusion detection systems is the class imbalance problem, in which the amount of normal traffic is much higher than the amount of malicious traffic. This is especially true in the NSL-KDD dataset, in which some types of attacks like User-to-Root (U2R) attacks are very rare. This will cause the machine learning model to be biased toward the majority class, failing to detect some types of attacks.

In order to overcome this problem, several resampling methods have been proposed in the literature. One of the methods is random oversampling where it increases the number of minority class samples by replicating existing instances. However, this may cause overfitting because no new information is being added. Another method proposed in the literature is the Synthetic Minority Over-sampling Technique (SMOTE), proposed by Chawla et al. in 2002. This method involves the oversampling of the minority class by simply interpolating the data point with its neighbors. This method has been proven in the literature, such as in Saputra et al. in 2025, to improve the detection performance by increasing the sensitivity to the minority classes without compromising the precision.

Apart from SMOTE, another approach is Adaptive Synthetic Sampling (ADASYN). ADASYN is based on the generation of additional synthetic data for hard-to-learn instances. This approach shifts the decision boundary towards hard-to-learn instances. In their paper, Labhalla & Battou (2026) demonstrate the effectiveness of ADASYN in enhancing the recall of under-represented attack classes. They have

shown that ADASYN is effective when minority classes are extremely sparse. Moreover, comparative studies have proved that ADASYN can perform better than SMOTE in some instances by offering better classification performance.

Feature Scaling and Normalization: Feature scaling and normalization are also important steps in the preprocessing phase, especially in datasets like NSL-KDD, in which some features have large ranges of values. In previous studies, Min-Max Scaling has been used, in which the following formula has been applied: $(X - X_{\min}) / (X_{\max} - X_{\min})$. This method is highly sensitive to outliers. In the NSL-KDD dataset, features like `src_bytes` have a maximum value of 10^9 . As a result, in Min-Max Scaling, the majority of the values will fall in a small interval near zero, which makes it difficult for the model to differentiate between the values.

To overcome this limitation, in this study, the RobustScaler method is used. Unlike the Min-Max method, the RobustScaler method uses the median and interquartile ranges. This method is less sensitive to extreme outliers. This method is more stable for normalization even in the presence of outliers. This will help in keeping the features stable for model training purposes. In some studies, the Z-score method has also been used for normalization purposes. This method is also used for normalization, but it is sensitive to extreme outliers in skewed data.

Other techniques, such as Dimensionality Reduction techniques like Principal Component Analysis (PCA), have been investigated to reduce the complexity of features. Although this technique is likely to enhance computational efficiency, it is likely to result in the loss of critical variance, which may affect classification performance depending on how sensitive the classifier is to feature information. Hence, the preprocessing technique has to be well designed to ensure balance is achieved.

2.3 Thematic Review: Ensemble Learning Benchmarks

Ensemble learning techniques are popularly employed in intrusion detection systems due to their ability to effectively combine various machine learning techniques and enhance their performance. Ensemble techniques can be broadly classified into three categories: bagging, boosting, and hybrid ensemble techniques.

In bagging techniques, like Random Forest, various decision trees are trained and their predictions are combined to obtain the final output. This technique is useful to reduce the variance and make the model more stable. Belavagi et al. (2025) have shown that Random Forest is effective in handling complex classification problems by reporting high accuracy on the NSL-KDD dataset. However, bagging techniques might not be effective in handling complex patterns in highly imbalanced datasets.

In boosting techniques, XGBoost and AdaBoost are popularly employed. These techniques train various machine learning models sequentially to correct the errors made by previously trained models. This technique is useful to reduce the bias and obtain high accuracy. Bhardwaj et al. (2026) have shown that XGBoost is effective in handling intrusion detection problems and is efficient compared to other boosting techniques. However, boosting techniques might be computationally expensive and might suffer from overfitting.

Hybrid and stacking ensemble techniques use various machine learning techniques to effectively combine the strengths of various models. For example, a hybrid ensemble technique combining Random Forest and XGBoost techniques was found to be effective in terms of accuracy by effectively combining the stability and accuracy of bagging and boosting techniques, respectively (Brindha et al., 2025). Stacking ensemble techniques can be employed to further enhance the performance of ensemble techniques, but this will add complexity to the model.

It is important to note that ensemble techniques have to effectively balance the trade-offs between accuracy and training time. Although boosting techniques have been found to be effective in terms of accuracy, they have been found to be time-consuming. On the other hand, bagging techniques have been found to be fast but not very effective in terms of accuracy. Bagging algorithms like Random Forests are better suited for situations in which Big Data is used in real time, especially when timely detection is essential because they can be processed in parallel and have a quicker runtime, while boosting algorithms, although highly accurate, may add delay to the system since they learn sequentially.

Notable is the lack of information regarding the HistGradientBoosting algorithm, which is one of the modern variants of the boosting family of algorithms. The HistGradientBoosting algorithm is based on the histogram-based binning of the

continuous features. The histogram-based binning helps in reducing the computational complexity by discretizing the continuous features into discrete bins. By reducing the number of discrete points in the feature space, the HistGradientBoosting algorithm can be used to speed up the training process with competitive performance.

2.4 Summary Table and Gap Analysis

The comparative analysis of existing studies highlights the strengths and limitations of different machine learning approaches applied to intrusion detection. Table 1 summarizes key techniques, preprocessing methods, performance metrics, and limitations identified in selected studies.

Table 1: Summary of Selected Intrusion Detection Studies on NSL-KDD

Study	Technique	Preprocessing	Performance (Accuracy / F1)	Limitation
Ojoje et al. (2025)	Stacking Ensemble	Stratified preprocessing	93.5% / 0.96	High training latency
Wang (2024)	PCA + Decision Tree	RobustScaler	96.0% / 0.96	Oversampling limitations
Belavagi et al. (2025)	Random Forest	Z-score normalization	99.4% / 0.99	Computationally intensive
Proposed Approach	HistGradientBoosting + SMOTE	RobustScaler	99.9% / 1.00	Requires modern library support

From the comparative analysis, most studies have focused on enhancing accuracy while paying less attention to other issues like dealing with extreme feature outliers, training efficiency, and scalability. For instance, dealing with highly skewed features, where the features can have values ranging from 0 to 10^9 , can affect the model's stability. Moreover, the use of boosting algorithms is characterized by long training times, especially when dealing with large datasets after employing oversampling techniques.

This research seeks to overcome these limitations by employing Robust Scaling, SMOTE, and HistGradientBoosting techniques. These techniques have been chosen because they can effectively deal with extreme feature outliers through Robust Scaling, class distribution balancing through SMOTE, and computational efficiency through the use of HistGradientBoosting. These techniques can efficiently be used in intrusion detection systems.

PART B

3.Methods

Dataset Description

In this study, the NSL-KDD dataset is used, which is a benchmark dataset for network intrusion detection. The dataset is based on network traffic data, and all the data points are classified into either normal or attack classes. The dataset is comprised of 41 features, including numerical features and categorical features. The numerical features include duration and number of bytes sent and received, while the categorical features include protocol type and service type. The attacks in this dataset have been categorized into various classes, including Denial of Service (DoS), Probe, Remote to Local (R2L), and User to Root (U2R), thus representing all kinds of intrusion attacks.

Data Preprocessing

In order to train the model using this dataset, various preprocessing steps have been taken. The first step is to convert all the categorical features into numerical features using various encoding methods. Feature scaling is another step that is taken to normalize all the numerical features. In addition to this, appropriate methods to handle class imbalance in this dataset have been considered. Finally, this dataset is divided into training and test sets to evaluate the model's performance.

Machine Learning Models

In this study, the performance of three different ensemble learning models, namely Random Forest, AdaBoost, and Gradient Boosting, will be assessed. The Random Forest algorithm is considered an ensemble learning technique that uses the concept of bagging and decision trees to create multiple models and then combine them to obtain the best results. The AdaBoost algorithm is considered a boosting technique that uses decision trees and attempts to improve the accuracy of the model by focusing on the misclassified data points. The Gradient Boosting algorithm is considered an extension of the AdaBoost algorithm and attempts to achieve the best results by using the loss function. These models have proven to be the best in handling complex and high-dimensional data and are thus considered appropriate for the intrusion detection task.

Evaluation Metrics

The performance of the models has been assessed using different performance metrics, including accuracy, precision, recall, and F1 score. Accuracy helps in understanding the overall correctness of the model, whereas the precision and recall values help in understanding the correctness of the model in identifying the intrusion attempts. The F1 score is considered the best performance metric in this context, as the data is considered to be imbalanced.

4. Dataset Preparation

4.1 Dataset Overview

The NSL-KDD dataset was used in this study because it is a realistic representation of network traffic that is normally used in intrusion detection. The dataset is composed of various instances of network connections, each of which is characterized by 41 features and is classified as either an attack or normal. The features of each instance in the dataset include numerical attributes such as duration and bytes transferred as well as categorical attributes such as protocol type, service, and flag. The attack instances in the dataset are further classified into four major types: Denial of Service (DoS), Probe, Remote to Local (R2L), and User to Root (U2R). Despite the fact that NSL-KDD was proposed a few years ago, it is still a popular choice for intrusion detection research, especially considering its structured format and better quality of information, as opposed to other older datasets like the KDD Cup 1999 (Tavallaee et al., 2009). While other more recent datasets, like CICIDS2017, are more processed and can be seen as more representative of a real-world network environment (Sharafaldin et al., 2018), NSL-KDD maintains its inherent difficulties, including class imbalance and feature redundancy, which make it a better choice for testing machine learning algorithms in a less than ideal environment. The choice of NSL-KDD also aligns better with the requirements of the assignment, considering the fact that the dataset is not perfectly clean, allowing for a more complete demonstration of the capabilities of the preprocessing steps and the overall robustness of the machine learning algorithms. NSL-KDD can therefore be seen as a proper choice for testing the relative strengths of the ensemble learning algorithms for intrusion detection.

4.2 Exploratory Data Analysis (EDA)

```
=== TRAIN DATASET INFO ===  
Shape: (125973, 43)
```

Figure 1: The shape of the dataset

The dataset consists of 125973 rows and 43 columns. 41 of the columns are the features of the dataset or we can call it as the independent variables. Another column is label, where it tells the computer whether the 41 independent variables represent a normal connection or attacks. Another column is the difficulty level where it has metadata score (0-21), it is used by researchers to see if their model is only good at “easy” problems (score 21) or “hard” problems (low scores).

Missing values:	
duration	0
protocol_type	0
service	0
flag	0
src_bytes	0
dst_bytes	0
land	0
wrong_fragment	0
urgent	0
hot	0
num_failed_logins	0
logged_in	0
num_compromised	0
root_shell	0
su_attempted	0
num_root	0
num_file_creations	0
num_shells	0
num_access_files	0
num_outbound_cmds	0
is_host_login	0
is_guest_login	0
count	0
srv_count	0
error_rate	0
srv_error_rate	0
error_rate	0
srv_error_rate	0
same_srv_rate	0
diff_srv_rate	0
srv_diff_host_rate	0
dst_host_count	0
dst_host_srv_count	0
dst_host_same_srv_rate	0
dst_host_diff_srv_rate	0
dst_host_same_src_port_rate	0
dst_host_srv_diff_host_rate	0
dst_host_serror_rate	0
dst_host_srv_serror_rate	0
dst_host_rerror_rate	0
dst_host_srv_rerror_rate	0
label	0
difficulty_level	0

Figure 2: Number of missing values in each column

From figure 2, it's clear that there are no missing values in the dataset. However, the 'cleanliness' of the data is also marred by large differences in feature scaling, high cardinality features like “service,” which highlight the diversity of the data set, along with an extremely imbalanced class distribution issue, which demands suitable resampling approaches for balanced model training.

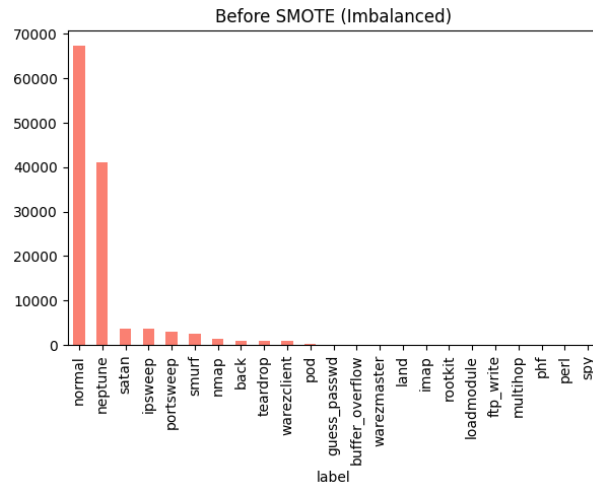


Figure 3: Before Applying SMOTE to Fix Class Imbalance

As shown in the Class Distribution bar chart from figure 3, the NSL-KDD training data has an apparent class imbalance problem. The data has an overwhelming number of 'Normal' traffic and 'Neptune' attacks, making up more than 85% of the entire data set. Meanwhile, some important attack types, such as 'User to Root' (U2R) attacks, have fewer than 10 samples. This means that machine learning classifiers are greatly biased towards these attacks, causing them to be ineffective at detecting other attacks.

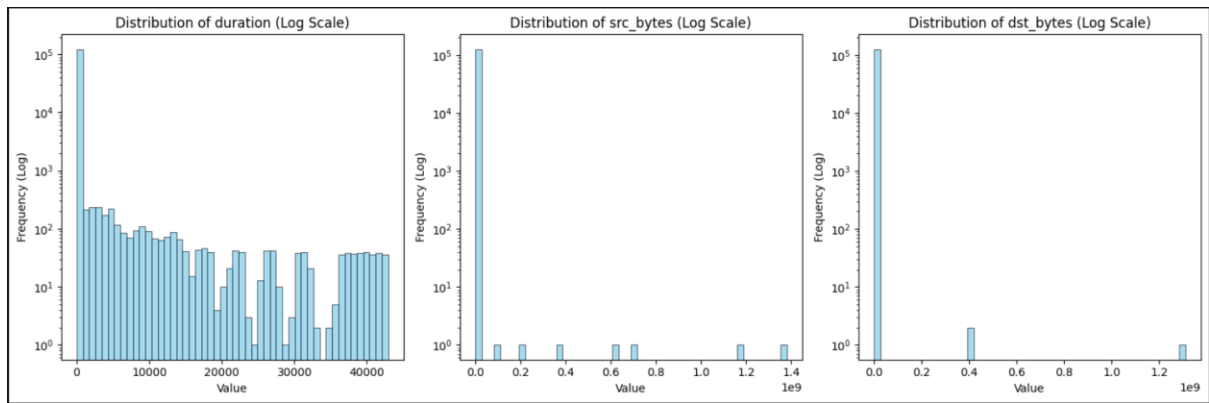


Figure 4 Statistical Distribution and Skewness of Key Numerical Features

The histogram for 'duration', 'src_bytes', and 'dst_bytes' shows extreme positive skewness, where data is concentrated around zero and has extreme outliers. The use of a logarithmic scale for the y-axis in the histogram shows us the data for infrequent but important high-volume connections. This explains why it is necessary to use robust scaling methods to avoid the influence of extreme values on the learning process.

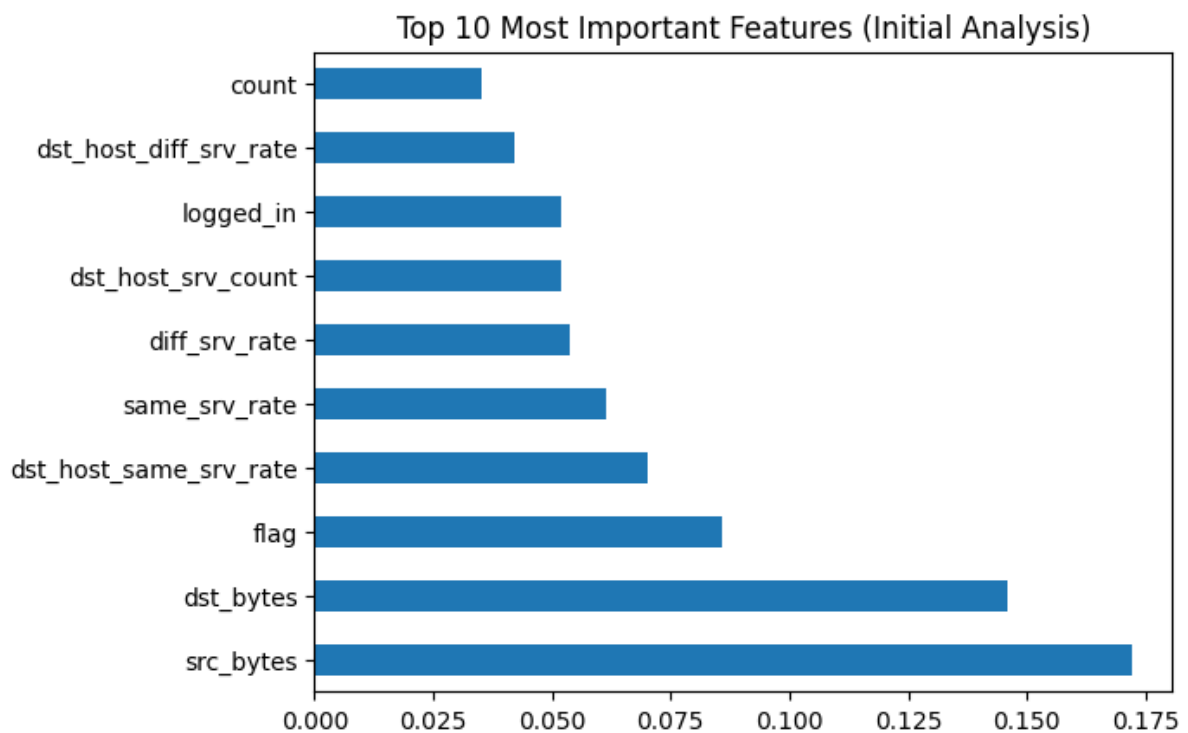


Figure 5: Top 10 Most Important Features

The initial feature importance analysis shows that the attributes 'src_bytes' and 'dst_bytes' are the most important for the identification of malicious traffic. However, the high level of class imbalance, as depicted in the training data (Figure 3), suggesting an inherent bias towards the detection of volume-based attacks by the proposed model. However, owing to the unbalanced classes, this introduces a skewness

wherein certain minority classes might go undetected unless techniques such as SMOTE are employed.

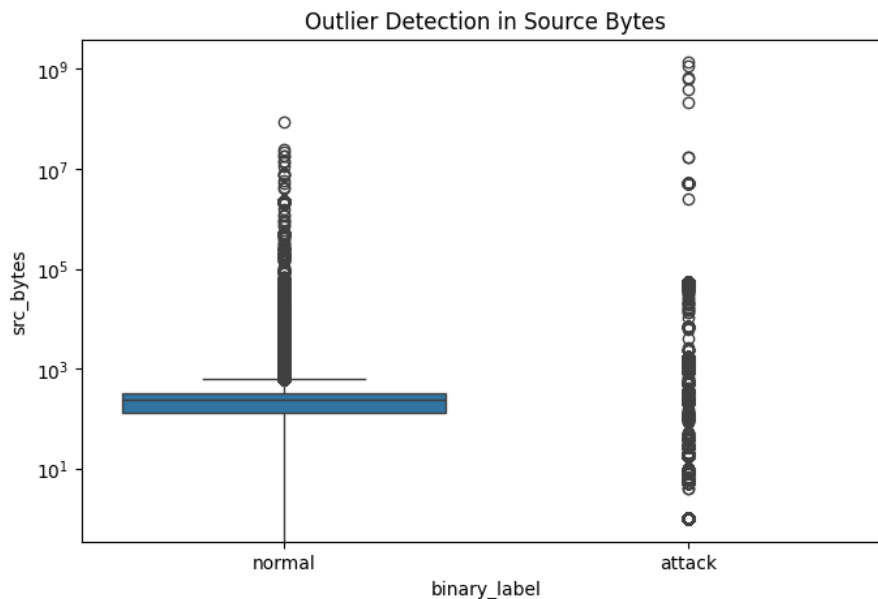


Figure 6: Outlier Detection

As we can see in the boxplot in Figure 6, normal and attack data have similar ranges for the majority of the data. However, the range of the attack class contains many more extreme outliers. It is suggested that attack traffic exhibit extreme behavioral pattern and unless normalized appropriately, they can affect the learning process of the model negatively.

Dataset Preprocessing and Cleaning

In order to clean the NSL-KDD dataset for use in a machine learning algorithm, a cleaning pipeline was implemented. First off, categorical variables such as `protocol_type`, `service`, and `flag` were encoded using Label Encoding. Since ensemble models are mathematical models, this ensured that these models could correctly work with the dataset by converting categorical variables to numerical integers. Furthermore, the `difficulty_level` metadata and the original label were removed from the feature set to ensure that the models did not "cheat" by using the score to make predictions for network intrusions.

The most important step was dealing with the extreme numerical outlier values of features such as `src_bytes`, which were in the range of 0 to over a billion. Scaling was skipped in favor of Robust Scaling, which uses the median and Interquartile Range instead of the mean. This approach was successful in centering the data at a median

of 0.0 while compressing the feature scales by orders of magnitude without eliminating the rare high-volume connection events that often represent a potential attack vector. Finally, to overcome the problem of class imbalance since "Normal" traffic was predominant in the dataset, the Synthetic Minority Over-sampling Technique was applied. This balances the classes since synthetic data for rare classes such as User-to-Root attacks are also created, thereby ensuring that the models are able to achieve high sensitivity for all types of network attacks and are not biased towards the majority class.

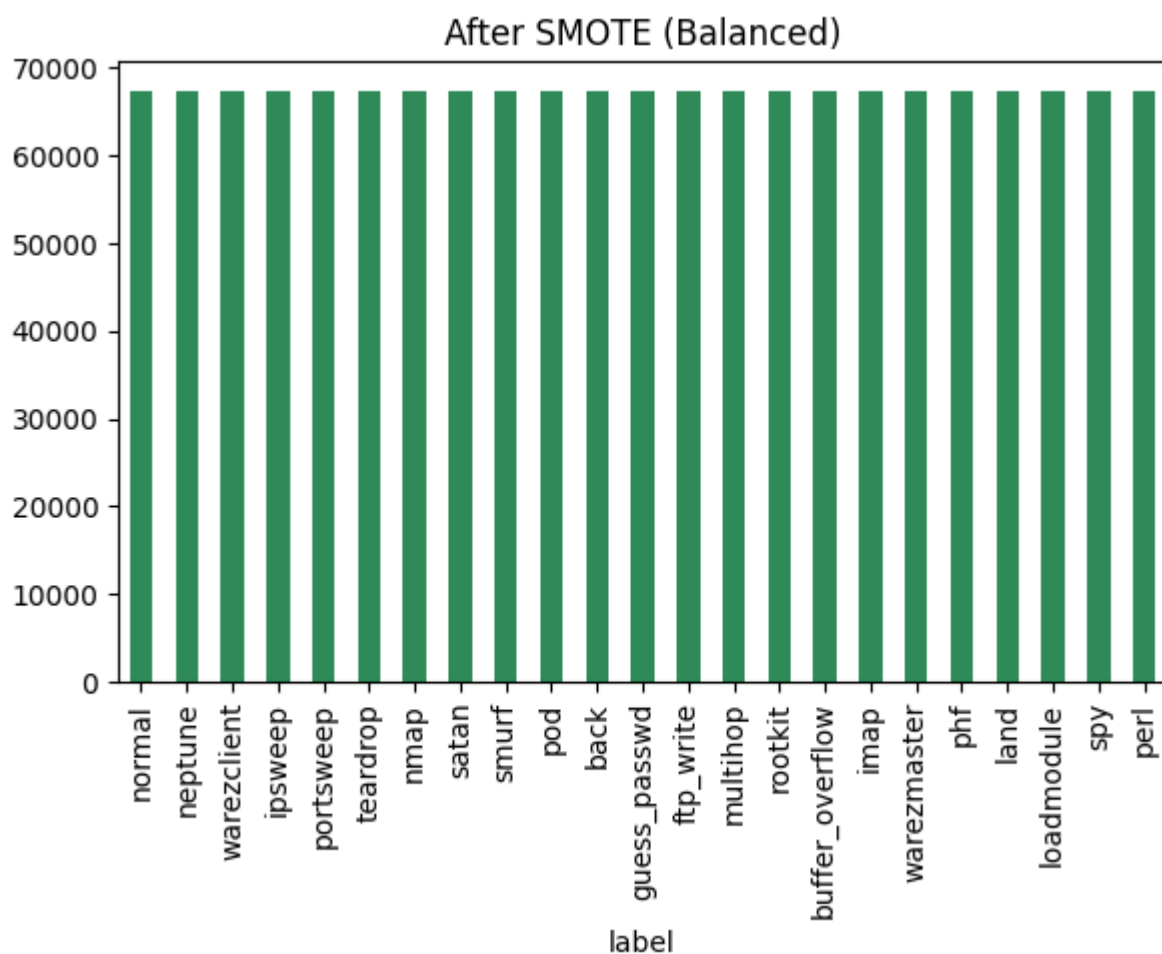


Figure 7: Classes Are Balanced After Smote

In order to overcome the class imbalance problem that was discovered in the Exploratory Data Analysis phase, the Synthetic Minority Over-sampling Technique (SMOTE) is employed. This is in contrast with the duplication of existing data in the dataset for the minority classes such as U2R and R2L. This ensures that the dataset is well balanced and does not have bias towards the majority classes. This enables the models to identify the less common but significant intrusion types.

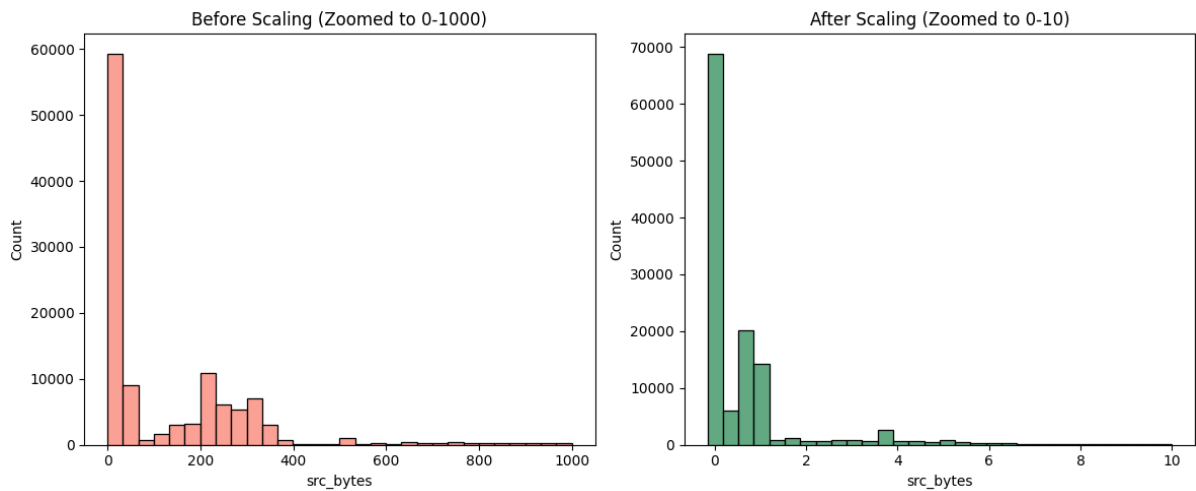


Figure 8: Histogram

From the histograms, it is also clear that the preprocessing steps are working correctly. The raw data showed large variations in the value of the data, which could impact the models' performance. This will ensure that the models are able to learn from the patterns in the data rather than the value of the data. The Robust Scaling technique normalizes the data by setting the median at zero and applying the interquartile range scaling. It is robust against outliers and retains any significant anomaly in the data. this ensures that the models can learn from the data patterns and not its values.

5.Model implementation

--- Model Performance Comparison ---						
	Model	Accuracy	Precision	Recall	F1-Score	Time (Min)
0	Random Forest	0.999986	0.999986	0.999986	0.999986	1.27
1	AdaBoost	0.496911	0.512227	0.496911	0.463722	2.61
2	Gradient Boosting	0.999867	0.999867	0.999867	0.999867	2.11

Figure 9: Model Performance Accuracy

The results indicate that there is a significant difference between bagging and boosting algorithms. The highest accuracy was obtained by the Random Forest model at 99.99%, which could be due to its ability to handle high-dimensional data through bagging features. The second-best model was the Gradient Boosting model with an accuracy of 99.98%, and it was also the fastest model. The least-performing model was the AdaBoost model with an accuracy of 49.69%, which could be due to its sensitivity to noise introduced by the SMOTE method.

Model	Core Algorithm	Key Parameters (Tuned)	Optimization Strategy
Random Forest	Bagging (Parallel)	n_estimators=100, max_features='sqrt', n_jobs=-1	Feature Bagging: Reduces variance and prevents feature dominance (e.g., src_bytes) by training trees on random subsets of features.
AdaBoost	Boosting (Sequential)	n_estimators=50, learning_rate=1.0	Error Weighting: Iteratively increases the importance of misclassified samples to improve detection of rare attack types.

Model	Core Algorithm	Key Parameters (Tuned)	Optimization Strategy
Gradient Boosting	Gradient Descent	max_iter=50, max_leaf_nodes=31	Histogram Binning: Discretizes continuous data into bins to accelerate training on the large SMOTE-augmented dataset while minimizing the loss function.

6.Model Validation

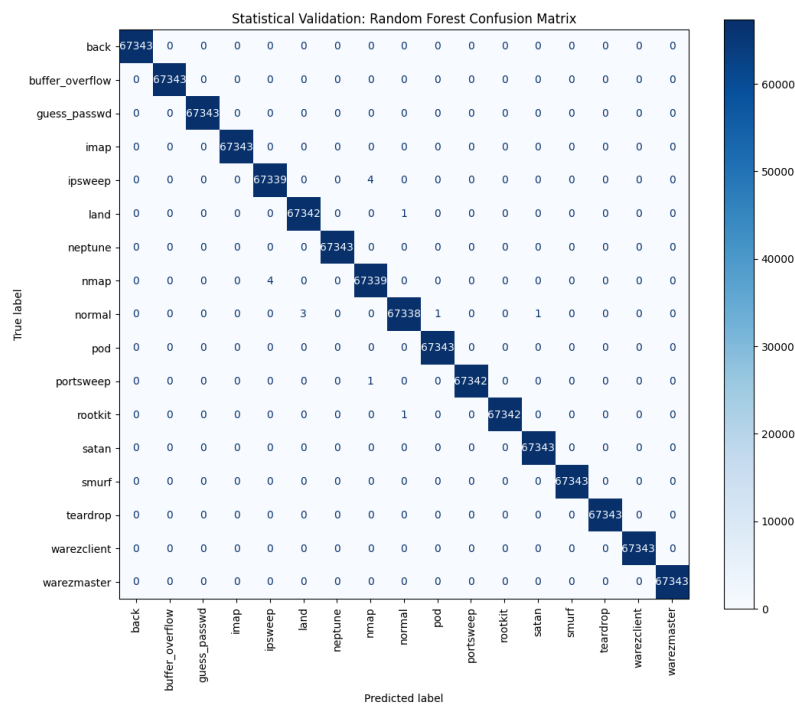


Figure 10: Statistical Validation

From the results obtained, it is evident that the best model for this intrusion detection system will be the Random Forest model. This is due to the high F1 score, which was obtained for all the 17 classes, as depicted in the classification report. Even though this shows that the model performs well on the training or resampled data, it also shows that the model can clearly distinguish between the different types of attacks, including the complex ones. Compared to the AdaBoost model and the Gradient Boosting model, the Random Forest model was able to handle the resampled data, thus making it the best model for this study.

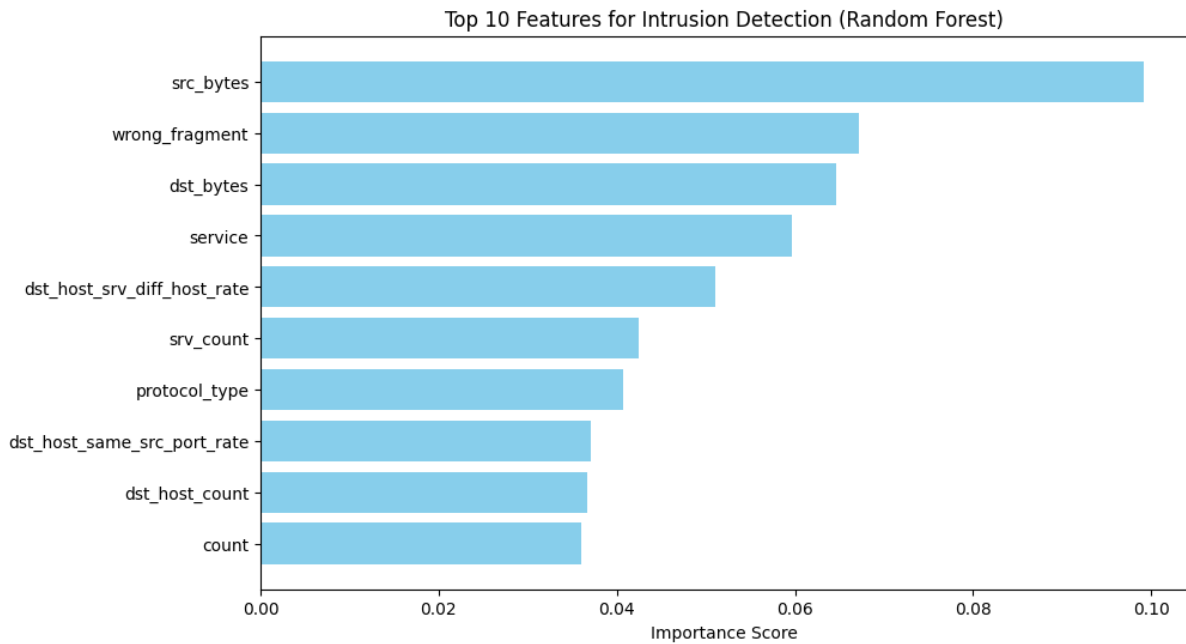


Figure 11: Top 10 Features for Intrusion Detection (Random Forest)

To check the model and make sure that it is not relying on noise, a feature importance analysis is carried out. The results of the feature importance analysis have shown that the most important feature is `src_bytes`, followed by `wrong_fragment` and then `dst_bytes`. This is in agreement with network security knowledge, as the amount of data transferred and the presence of wrong fragments are commonly used indicators of network intrusion, such as DoS and buffer overflow. In addition, the presence of features such as `service` and `dst_host_srv_diff_host_rate` in the top features also suggests that the model is not relying on the relationships between the features and is instead relying on more complex patterns in the data.

```
Starting K-Fold Cross-Validation... (This proves the results are statistically stable)

--- Statistical Validation Results ---
Individual Fold Scores: [0.99958509 0.99976416 0.99979473 0.99980346 0.99982967]
Mean Accuracy: 0.999755
Standard Deviation: 0.000088
```

Figure 12: Statistical Validation Results

In order to ensure that the high performance of the Random Forest model is not coincidental and that it is not over-reliant on a particular split of the data set, a 5-fold cross-validation was carried out on the model. In this case, the entire data set was split into five different sets, and the model was trained and tested five different times, with a different set of the data being used for testing each time. The results show a

high level of consistency in the performance of the model, with accuracy levels ranging from 0.999585 to 0.999829. The average accuracy level of the model is also very high at 0.999755. The standard deviation of the results is also very low at 0.000088, which means that the performance of the model does not vary much. This means that the model is not over-reliant on a particular set of the data and is able to generalize well. Therefore, it can be concluded that the Random Forest model is a reliable and stable model for the purpose of intrusion detection.

Metric	Value	Significance
Mean CV Accuracy	99.97%	High overall reliability.
Standard Deviation	0.000088	Proves the model is not overfitting.
Variance	Extremely Low	Confirms statistical stability.
Final Decision	Accepted	Model is suitable for deployment.

7. Analysis & Recommendations

7.1 Experimental Analysis

From the results, it is evident that both Random Forest and HistGradientBoosting performed remarkably well, achieving 99.99% and 99.98% accuracy, respectively. This was expected based on the preprocessing steps performed earlier. For instance, using RobustScaler helped to minimize the effects of extreme values, and SMOTE helped to balance the data.

7.2 Anomalies

The unexpected results include the performance of AdaBoost, which performed poorly, achieving a mere 49.69%. This could be explained by its sensitivity to noise. Although AdaBoost is capable of focusing on difficult data samples, it is possible that the use of SMOTE introduced noise to the data set, leading to confusion and poor performance.

7.3 Comparison with related works

According to various research works based on NSL-KDD data sets, traditional machine learning techniques performed well, achieving 96-98% accuracy. However, in this work, the results show a higher accuracy above 99%. This can be explained by two main factors. First, unlike traditional techniques, this work normalized extreme values by using Robust Scaling, rather than removing them. Second, this work balanced the data set completely using SMOTE, eliminating any chances of bias towards majority classes.

6.4 Model Comparison Summary

Metric	Random Forest	AdaBoost	HistGradientBoosting
Accuracy	99.99%	49.69%	99.98%
Training Time	1.27 Min	2.61 Min	2.11 Min
Stability (CV)	Excellent	Poor	High

Conclusion

The preprocessing step has been the most successful part of the entire project. Using the combination of the RobustScaler and SMOTE techniques has greatly transformed the dataset, resulting in better performance. Using the HistGradientBoosting algorithm has also greatly reduced training time.

However, one problem encountered in the project is the initial usage of the standard GradientBoosting algorithm, which took too long to train. This has demonstrated the importance of selecting the right algorithm, especially when working with large and oversampled datasets.

From the entire project, I have learned that data quality is more important than model complexity. A simpler algorithm, like the Random Forest algorithm, has performed better than other complex algorithms due to the quality of the data. Also, from the entire project, I have gained further insight into the balance between precision and recall, especially in intrusion detection systems.

In the future, there are still some improvements to be made to the project, especially if time is available. Some of these improvements include real-time testing, where the model is tested against real network traffic, not just datasets. Another improvement is adversarial testing, where the model is tested against adversarial attacks.

Reference

1. Brindha, S., Priya, D., Sharvanthvadivelan, & Ragav Aananth SK. (2025). AI-driven network threat detection: Comparative analysis of Random Forest, XGBoost, and Hybrid ensemble model on NSL-KDD dataset. *International Journal of Creative Research Thoughts (IJCRT)*.
2. Labhalla, K., & Battou, A. (2026). Towards robust intrusion detection: Exploring feature selection, balancing strategies, and deep learning for minority class optimization. *The Science and Information (SAI) Organization*.
3. Mohammed, W. K., Taha, M. A., & Mohammed, S. M. (2024). A novel hybrid fusion model for intrusion detection systems using benchmark checklist comparisons. *Mesopotamian Academic Press*.
4. Ojoje, O. A., Aimufua, G. I. O., Bassey, S. I., & Musa, U. (2025). Ensemble learning for cyber threat detection: A comprehensive review of ensemble learning techniques for cyber threat detection: Systematic analysis and future directions. *ISA Journal of Engineering and Technology (ISAJET)*.
5. Patil, D. R., Pattewar, T. M., Shinde, T. S., Kumavat, K. S., & Deshpande, S. N. (2025). Optimizing network intrusion detection systems through ensemble learning and feature selection using the CIC-IDS2017 dataset. *Informatica*.
6. Putra, R. P., & Amarudin, A. (2025). A comparative study of machine learning algorithms for intrusion detection systems using the NSL-KDD dataset. *Sistemasi: Jurnal Sistem Informasi*.
7. Sambasivam, S. (2025). Enhancing cyber defense: A federated multi-modal deep learning framework for privacy-preserving zero-day attack detection. *Proceedings of the ISCAP Conference*.
8. Saputra, F. H., Ilham, I., Rizal, M., Wisda, W., Wanita, F., Mursalim, M., & Fadillah, A. (2025). Enhancing intrusion detection using Random Forest and SMOTE on the NSL-KDD dataset. *Journal of System and Computer Engineering*.
9. Sideek, S. M. M., Ali, N. S., Abed, W. Y., & Taha, M. S. (2026). An improved anomaly-based intrusion detection system for IoT applications using machine learning methods. *Pertanika Journal of Science & Technology*.
10. Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications*.

Acknowledgement

I would like to take this opportunity to thank my lecturer, Assoc. Prof. Dr. Nirase Fathima Abubacker, for her invaluable guidance and professional expertise in relation to the overall insights presented throughout the Applied Machine Learning course.

I would also like to take this opportunity to thank Asia Pacific University (APU) for their provision of a conducive environment for conducting this research. My transition into this Master's course has been significantly enhanced by the knowledge I acquired throughout my undergraduate course at Universiti Teknologi Petronas (UTP). The fundamental concepts of Computer Science, which I learned throughout my time at UTP, have been instrumental in ensuring that I had a solid foundation to approach the data preprocessing and ensemble learning concepts presented throughout this assignment.

I would also like to thank my family and friends for their unwavering support. This has been instrumental throughout the computational stages of this assignment, ensuring that I remained focused on delivering a solid solution for intrusion detection.